Solution: Range Sum of BST (easy)

Problem Statement

Given a Binary Search Tree (BST) and a range defined by two integers, L and R, calculate the sum of all the values of nodes that fall within this range. The node's value is inclusive within the range if and only if L <= node's value <= R.

Examples:

Example 1:

Input:

```
Tree:
    10
   /  \
  5   15
 / \    \
3   7   18
Range: [7, 15]
```

Expected Output: 32

Justification: The values that fall within the range [7, 15] are 7, 10, and 15. Their sum is 7 + 10 + 15 = 32.

Example 2:

Input:

```
Tree:
    20
   /  \
  8   25
 / \
3   10
Range: [3, 10]
```

Expected Output: 18

Justification: The values within the range [3, 10] are 3, 5, and 10. Their sum is 3 + 5 + 10 = 18.

Example 3:

Input:

```
Tree:
    30
      \
       35
      /
     32
Range: [30, 34]
```

Expected Output: 62

Justification: The values within the range [30, 34] are 30 and 32. Their sum is 30 + 32 = 62.

Solution

To solve this problem, start traversing the BST from the root node and accumulate the sum of node values that fall within the specified range (low to high). If the current node is null, return 0, as there's nothing to add. For nodes within the range, add their value to the sum. Since it's a BST, if the current node's value is greater than high, there's no need to explore its right subtree, as all values there will be greater. Similarly, if the current node's value is less than low, ignore its left subtree. This selective traversal reduces the number of nodes visited, optimizing the process.

1. **Starting Point**: Start with the root of the BST.
2. **Range Check**: For every node you encounter, check if its value falls within the range [L, R].
3. **Inclusion**: If the value is within the range, include it in our sum.
4. **Traversal Decisions**: Utilize the properties of BST for traversal decisions:
   - If the current node's value is greater than L, we should check its left child because there might be values in the left subtree that fall within the range.
   - If the current node's value is less than R, we should check its right child as there might be values in the right subtree that fall within the range.
5. **Summation**: Keep a running sum of all node values that are within the specified range.

This approach ensures we don't traverse parts of the BST that don't contain values within our range, optimizing the process.

Algorithm Walkthrough:

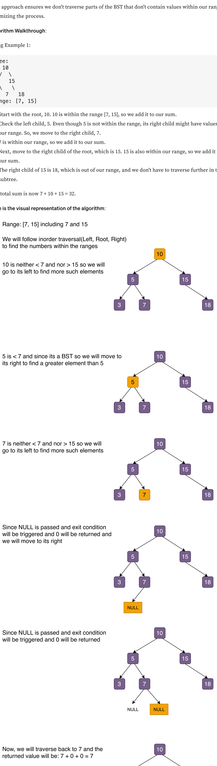
Using Example 1:

```
Tree:
    10
   /  \
  5   15
 / \    \
3   7   18
Range: [7, 15]
```

- Start with the root, 10. 10 is within the range [7, 15], so we add it to our sum.
- Check the left child, 5. Even though 5 is not within the range, its right child might have values in our range. So, we move to the right child, 7.
- 7 is within our range, so we add it to our sum.
- Next, move to the right child of the root, which is 15. 15 is also within our range, so we add it to our sum.
- The right child of 15 is 18, which is out of our range, and we don't have to traverse further in this subtree.

The total sum is now 7 + 10 + 15 = 32.

Here is the visual representation of the algorithm:



Code

Here is the code for this algorithm:

```
Java
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

public class Solution {

    public int rangeSumBST(TreeNode root, int L, int R) {
        if (root == null) return 0; // Base case

        // If the current node's value is out of the range on the higher side
        if (root.val > R) return rangeSumBST(root.left, L, R);

        // If the current node's value is out of the range on the lower side
        if (root.val < L) return rangeSumBST(root.right, L, R);

        // Current node's value is in the range, include it and check both children
        return root.val + rangeSumBST(root.left, L, R) + rangeSumBST(root.right, L, R);
    }
}
```

Run Code Result | Stdout

You must run your code first

Execute

Complexity Analysis

Time Complexity: O(N) - In the worst case, we might have to traverse all the nodes in the BST. Space Complexity: O(H) - Where H is the height of the BST. This space is used by the call stack during the recursive calls.